

# Handbook CHACOTEAM!

CHACOTEAM! — UTFSM

Última actualización: September 23, 2026

## Contents

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| <b>1</b> | <b>Template Base</b>                             | <b>1</b>  |
| <b>2</b> | <b>Grafos</b>                                    | <b>1</b>  |
| 2.1      | Dijkstra – $O((V + E) \log V)$                   | 1         |
| 2.2      | Floyd-Warshall – $O(V^3)$                        | 1         |
| 2.3      | Bellman-Ford – $O(VE)$                           | 2         |
| 2.4      | Kruskal – $O(E \log E)$                          | 2         |
| 2.5      | LCA – $O(V \log V)$ build, $O(\log V)$ query     | 2         |
| 2.6      | Union-Find / DSU – $O(\alpha(n))$                | 2         |
| <b>3</b> | <b>Flujo y Conectividad</b>                      | <b>3</b>  |
| 3.1      | Dinic                                            | 3         |
| 3.2      | 2-SAT                                            | 3         |
| 3.3      | Camino-dirigido-euler                            | 3         |
| 3.4      | Camino-nodirigido-euler                          | 4         |
| 3.5      | Tarjan                                           | 4         |
| 3.6      | tarjan SCC                                       | 4         |
| <b>4</b> | <b>Estructuras de Datos</b>                      | <b>5</b>  |
| 4.1      | Segment Tree – $O(\log n)$                       | 5         |
| 4.2      | Sparse Table – $O(n \log n)$ build, $O(1)$ query | 5         |
| 4.3      | Segment Tree Lazy – $O(\log n)$                  | 5         |
| 4.4      | Fenwick Tree – $O(\log n)$                       | 6         |
| 4.5      | Fenwick Tree 2D – $O(\log n)$                    | 6         |
| 4.6      | Line container – $O(\log n)$                     | 6         |
| 4.7      | Sqrt decomp – $O(\sqrt{n})$                      | 7         |
| <b>5</b> | <b>Matemáticas</b>                               | <b>7</b>  |
| 5.1      | Template                                         | 7         |
| 5.2      | BinPow                                           | 7         |
| 5.3      | CatalanNumbers                                   | 7         |
| 5.4      | ConvexHull                                       | 7         |
| 5.5      | Chinese Remainder Theorem                        | 8         |
| 5.6      | Discrete Log                                     | 8         |
| 5.7      | Discrete Root                                    | 8         |
| 5.8      | Erathostenes Sieve                               | 8         |
| 5.9      | Factorization using Pollard Rho                  | 8         |
| 5.10     | Fast Fourier Transform                           | 9         |
| 5.11     | Gaussian Elimination                             | 9         |
| 5.12     | GCD                                              | 9         |
| 5.13     | GCD Extended                                     | 10        |
| 5.14     | Geometry Structures                              | 10        |
| 5.15     | Inclusion Exclusion                              | 11        |
| 5.16     | Miller Rabin                                     | 11        |
| 5.17     | Mobius                                           | 11        |
| 5.18     | Number Theoric Transform                         | 11        |
| 5.19     | Primitive Root (Generator of a Prime Group)      | 12        |
| 5.20     | Simpson Rule                                     | 12        |
| 5.21     | Tonelli Shanks                                   | 12        |
| 5.22     | Euler Totiente                                   | 12        |
| <b>6</b> | <b>Strings</b>                                   | <b>12</b> |
| 6.1      | KMP – $O(n + m)$                                 | 12        |
| 6.2      | Z-Function – $O(n + m)$                          | 13        |
| 6.3      | Trie – $O( s )$ por operación                    | 13        |
| 6.4      | Rolling Hashing                                  | 13        |
| 6.5      | aho-corasick                                     | 13        |

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>7</b> | <b>Programación Dinámica</b>                                  | <b>14</b> |
| 7.1      | Longest Increasing Subsequence – $O(n \log n)$                | 14        |
| 7.2      | LIS of length K – $O(nk \log n)$                              | 14        |
| 7.3      | LIS elements participate of Optimal solutions – $O(n \log n)$ | 14        |
| 7.4      | Knapsack 0/1 – $O(nW)$                                        | 15        |
| <b>8</b> | <b>Misceláneos</b>                                            | <b>15</b> |
| 8.1      | Búsqueda Binaria – $O(\log n)$                                | 15        |

## 1 Template Base

```

1 #pragma GCC optimize("O3", "unroll-loops")
2 #include <bits/stdc++.h>
3 using namespace std;
4 using ll = long long;
5 #define all(v) (v).begin(), (v).end()
6
7 void solve(){
8
9 }
10
11 int main(){
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14     ll t = 1;
15     cin >> t;
16     while(t--){
17         solve();
18     }
19     return 0;
20 }
21

```

template.cpp

## 2 Grafos

### 2.1 Dijkstra – $O((V + E) \log V)$

```

1 vector<ll> dist;
2 vector<vector<pair<ll,ll>>> g;
3
4 void dijkstra(){
5     priority_queue<pair<ll,ll>, vector<pair<ll,ll>>,
6         greater<pair<ll,ll>>> pq;
7     pq.push({0,0});
8     while(pq.size()){
9         ll c, node;
10        auto p = pq.top(); pq.pop();
11        c = p.first, node = p.second;
12        if(dist[node] > c){
13            dist[node] = c;
14            for(auto [c_, v_] : g[node]){
15                if(dist[v_] > c + c_) {
16                    pq.push({c+c_, v_});
17                }
18            }
19        }
20 }

```

dijkstra.cpp

### 2.2 Floyd-Warshall – $O(V^3)$

```

1 /*
2 *Description:* All pairs shortest path,  $O(V^3)$ 
3 *Status:* Tested
4 */
5 for (int i = 1; i <= n; i++) {
6     for (int j = 1; j <= n; j++) {
7         if (i == j) distance[i][j] = 0;
8         else if (adj[i][j]) distance[i][j] = adj[i][j];
9         else distance[i][j] = INF;
10    }
11 }
12 for (int k = 1; k <= n; k++) {
13     for (int i = 1; i <= n; i++) {
14         for (int j = 1; j <= n; j++) {
15             distance[i][j] = min(distance[i][j],
16                 distance[i][k]+distance[k][j]);
17         }
18     }
19 }

```

```

18 }
19 }
20

```

Floyd-Warshall.cpp

## 2.3 Bellman-Ford – $O(VE)$

```

1  /*
2  *Description:* Calculates shortest paths from $s$ in a
3  *graph that might have negative edge weights.
4  *Status:* Tested on CSES
5  */
6  struct BellmanFord {
7      struct Edge { int from, to; ll weight; };
8      int n, last_updated = -1; const ll INF = 1e18;
9      vector<int> p; vector<ll> dist;
10     BellmanFord(vector<Edge> &G, int s, int _n) {
11         n = _n; dist.assign(n+1, INF);
12         p.assign(n+1, -1); dist[s] = 0;
13         for (int i = 1; i <= n; i++) {
14             last_updated = -1;
15             for (Edge &e : G)
16                 if (dist[e.from] != INF && dist[e.from] + e.weight
17                     < dist[e.to]) {
18                     dist[e.to] = dist[e.from] + e.weight;
19                     p[e.to] = e.from; last_updated = e.to;
20                 }
21         }
22     }
23     bool getCycle(vector<int> &cycle) {
24         if (last_updated == -1) return false;
25         for (int i = 0; i < n-1; i++)
26             last_updated = p[last_updated];
27         for (int x = last_updated ;; x = p[x]) {
28             cycle.push_back(x);
29             if (x == last_updated and cycle.size() > 1) break;
30         }
31         reverse(cycle.begin(), cycle.end());
32         return true;
33     }
34 };
35

```

Bellman-Ford.cpp

## 2.4 Kruskal – $O(E \log E)$

```

1  /*
2  *Description:* Minimum spanning tree in  $O(E \log E)$ 
3  *Status:* Tested
4  */
5  struct Edge {
6      int a; int b; ll w;
7      Edge(int a_, int b_, ll w_) : a(a_), b(b_), w(w_) {}
8  };
9  bool c_edge(Edge &a, Edge &b) { return a.w < b.w; }
10 ll Kruskal() {
11     int n = G.size();
12     union_find sets(n);
13     vector<Edge> edges;
14     for (int i = 0; i < n; i++) {
15         for (auto eg : G[i]) {
16             // node i to node eg.first with cost eg.second
17             edges.emplace_back(i, eg.first, eg.second);
18         }
19     }
20     sort(edges.begin(), edges.end(), c_edge);
21     ll min_cost = 0;
22     for (Edge e : edges) {
23         if (sets.sameSet(e.a, e.b) != true) {
24             tree.emplace_back(e.a, e.b, e.w);
25             min_cost += e.w;
26             sets.unionSet(e.a, e.b);
27         }
28     }
29     return min_cost;
30 }
31

```

Kruskal.cpp

## 2.5 LCA – $O(V \log V)$ build, $O(\log V)$ query

```

1  /*
2  *Description:* Lowest Common Ancestor using binary
3  *lifting.
4  *Preprocessing:  $O(V \log V)$ 
5  *Query:  $O(\log V)$ 

```

```

5  *Works on rooted trees and forests represented by
6  *adjacency lists.
7  *Status:* Tested
8  */
9  struct LCA {
10     vector<vector<ll>> T, parent;
11     vector<ll> depth, vis, comp;
12     ll LOGN, V;
13     LCA(vector<vector<ll>> &T) : T(T) {
14         V = (ll)T.size();
15         LOGN = 1;
16         while ((1LL << LOGN) <= max(1LL, V))
17             LOGN++;
18         parent.assign(V, vector<ll>(LOGN, 0));
19         depth.assign(V, 0);
20         vis.assign(V, 0);
21         comp.assign(V, -1);
22         ll id = 0;
23         for (ll u = 0; u < V; u++) {
24             if (!vis[u]) {
25                 dfs(u, u, id++);
26             }
27         }
28     }
29     void dfs(ll u, ll p, ll id) {
30         vis[u] = 1;
31         comp[u] = id;
32         parent[u][0] = p;
33         for (ll j = 1; j < LOGN; j++)
34             parent[u][j] = parent[parent[u][j-1]][j-1];
35     }
36     for (ll v : T[u]) {
37         if (!vis[v]) {
38             depth[v] = depth[u] + 1;
39             dfs(v, u, id);
40         }
41     }
42 }
43 ll query(ll u, ll v) {
44     if (comp[u] != comp[v])
45         return -1;
46     if (depth[u] < depth[v])
47         swap(u, v);
48     ll k = depth[u] - depth[v];
49     for (ll j = LOGN - 1; j >= 0; j--) {
50         if ((k >> j) & 1)
51             u = parent[u][j];
52     }
53     if (u == v)
54         return u;
55     for (ll j = LOGN - 1; j >= 0; j--) {
56         if (parent[u][j] != parent[v][j]) {
57             u = parent[u][j];
58             v = parent[v][j];
59         }
60     }
61     return parent[u][0];
62 }
63 };
64

```

LCA.cpp

## 2.6 Union-Find / DSU – $O(\alpha(n))$

```

1  struct union_find {
2      vector<int> e;
3      union_find(int n) { e.assign(n, -1); }
4      int findSet(int x) {
5          return (e[x] < 0
6              ? x : e[x] = findSet(e[x]));
7      }
8      bool sameSet(int x, int y) {
9          return findSet(x) == findSet(y);
10     }
11     int size(int x) { return -e[findSet(x)]; }
12     bool unionSet(int x, int y) {
13         x = findSet(x), y = findSet(y);
14         if (x == y) return 0;
15         if (e[x] > e[y]) swap(x, y);
16         e[x] += e[y], e[y] = x;
17         return 1;
18     }
19 };
20

```

dsu.cpp

## 3 Flujo y Conectividad

### 3.1 Dinic

```

1  /*
2  *Author:* Pablo Messina
3  *Source:* https://github.com/PabloMessina/Competitive-Programming-Material
4  *Description:* Flow algorithm with complexity  $\mathcal{O}(|E| \cdot |V|^2)$ 
5  *Status:* Not tested
6  */
7  template<class T>
8  struct Dinic {
9      struct Edge { int to, rev; T f, c; bool r; };
10     int n, t_; vector<vector<Edge>> G;
11     vector<int> D, q, W;
12     bool bfs(int s, int t) {
13         W.assign(n, 0); D.assign(n, -1); D[s] = 0;
14         int f = 0, l = 0; q[l++] = s;
15         while (f < l) {
16             int u = q[f++];
17             for (const Edge &e : G[u]) if (D[e.to] == -1 && e.f < e.c)
18                 D[e.to] = D[u]+1, q[l++] = e.to;
19         }
20         return D[t] != -1;
21     }
22     int dfs(int u, T f) {
23         if (u == t_) return f;
24         for (int &i = W[u]; i < (int)G[u].size(); ++i) {
25             Edge &e = G[u][i]; int v = e.to;
26             if (e.c <= e.f || D[v] != D[u] + 1) continue;
27             T df = dfs(v, min(f, e.c - e.f));
28             if (df > 0) { e.f += df, G[v][e.rev].f -= df;
29                 return df; }
30         }
31         return 0;
32     }
33     Dinic(int N) : n(N), G(N), D(N), q(N) {}
34     void add_edge(int u, int v, T cap) {
35         G[u].push_back({v, (int)G[v].size(), 0, cap, 0});
36         G[v].push_back({u, (int)G[u].size()-1, 0, 0, 1}); //
37         // Use cap instead of 0 if bidirectional
38     }
39     T max_flow(int s, int t) {
40         t_ = t; T ans = 0;
41         while (bfs(s, t)) while (T dl = dfs(s, numeric_limits<T>::max())) ans += dl;
42         return ans;
43     }
44 }
45
46 /*
47 Para min cut
48 for(auto [u, v] : edges){
49     if((g.D[u] != -1) != (g.D[v] != -1)){
50         ans.push_back({u,v});
51     }
52 }
53 */

```

dinic.cpp

### 3.2 2-SAT

```

1  struct TwoSat {
2      int N;
3      vector<vector<int>> gr;
4      vector<int> values;
5      TwoSat(int n = 0) : N(n), gr(2*n) {}
6      int addVar() {
7          gr.emplace_back(); gr.emplace_back();
8          return N++;
9      }
10     void either(int f, int j) {
11         f = max(2*f, -1-2*f);
12         j = max(2*j, -1-2*j);
13         gr[f].push_back(j^1);
14         gr[j].push_back(f^1);
15     }
16     void setValue(int x) { either(x, x); }
17     void atMostOne(const vector<int>& li) {
18         if ((int)li.size() <= 1) return;
19         int cur = ~li[0];
20         for (int i = 2; i < (int)li.size(); i++) {
21             int next = addVar();
22             either(cur, ~li[i]);
23             either(cur, next);
24             either(~li[i], next);
25             cur = ~next;

```

```

26         }
27         either(cur, ~li[i]);
28     }
29     vector<int> val, comp, z; int Time = 0;
30     int dfs(int i) {
31         int low = val[i] = ++Time, x; z.push_back(i);
32         for (int e : gr[i]) if (!comp[e])
33             low = min(low, val[e] ? val[e] : dfs(e));
34         if (low == val[i]) do {
35             x = z.back(); z.pop_back();
36             comp[x] = low;
37             if (values[x]>1) == -1)
38                 values[x]>1 = x&1;
39         } while (x != i);
40         return val[i] = low;
41     }
42     bool solve() {
43         values.assign(N, -1);
44         val.assign(2*N, 0); comp = val;
45         for (int i = 0; i < 2*N; i++) if (!comp[i]) dfs(i);
46         for (int i = 0; i < N; i++)
47             if (comp[2*i] == comp[2*i+1]) return false;
48         return true;
49     }
50 };

```

2-SAT.cpp

### 3.3 Camino-dirigido-euler

```

1  /*
2  *Description:* Finds an Eulerian trail/circuit in a
3  directed graph using Hierholzer's algorithm. Handles
4  self-loops and parallel edges.
5  - A trail exists iff all vertices satisfy outdeg = indeg
6  (circuit), or exactly one has outdeg - indeg = 1 (
7  start) and one has indeg - outdeg = 1 (end), and the
8  edge-set is weakly connected.
9  - 'vertices': vertex sequence of size $m+1$.
10 - 'edges': edge index sequence of size $m$.
11 *Complexity:*  $\mathcal{O}(V + E)$ 
12 *Status:* Tested on https://judge.yosupo.jp/problem/eulerian\_trail\_directed
13 */
14 struct directed_euler_trail {
15     ll n, m = 0;
16     vector<vector<ll>> adj;
17     vector<pair<ll, ll>> E;
18     vector<ll> indeg, outdeg, vertices, edges;
19     directed_euler_trail(ll n) : n(n), adj(n), indeg(n),
20         outdeg(n) {}
21     void add_edge(ll u, ll v) {
22         adj[u].push_back(m);
23         E.push_back({u, v});
24         outdeg[u]++, indeg[v]++, m++;
25     }
26     ll find_start() {
27         ll s = -1, cp = 0, cm = 0;
28         for (ll v = 0; v < n; v++) {
29             ll d = outdeg[v] - indeg[v];
30             if (d == 1) cp++, s = v;
31             else if (d == -1) cm++;
32             else if (d != 0) return -2;
33             if (outdeg[v] > 0 && s < 0) s = v;
34         }
35         if (cp == 0 && cm == 0) return s;
36         return (cp == 1 && cm == 1) ? s : -2;
37     }
38     bool solve() {
39         if (m == 0) { vertices = {0}; return true; }
40         ll s = find_start();
41         if (s == -2) return false;
42         vector<ll> ep(n);
43         edges.resize(m);
44         ll pp = 0, pq = m, v = s;
45         while (pp < pq) {
46             if (ep[v] >= (ll)adj[v].size()) {
47                 if (pp == 0)
48                     return false;
49                 edges[--pq] = edges[--pp];
50                 v = E[edges[pq]].first;
51             } else {
52                 ll e = adj[v][ep[v]++];
53                 edges[pp++] = e;
54                 v = E[e].second;
55             }
56         }
57         vertices = {s};
58         for (ll e : edges) vertices.push_back(E[e].second);
59         return true;
60     }
61 };

```

## CaminoDirigidoEuler.cpp

## 3.4 Camino-nodirigido-euler

```

1  /*
2  *Description:* Finds an Eulerian trail/circuit in an
3  undirected graph using Hierholzer's algorithm.
4  Handles self-loops and parallel edges.
5  - A trail exists iff 0 or 2 vertices have odd degree and
6  the graph is connected.
7  - Uses XOR trick: 'v ^= E[e].first ^ E[e].second' to
8  traverse edges.
9  - 'vertices': vertex sequence of size $m+1$.
10 - 'edges': edge index sequence of size $m$.
11 *Complexity:* $O(V + E)$
12 *Status:* Tested on https://judge.yosupo.jp/problem/eulerian\_trail\_undirected
13 */
14 struct undirected_euler_trail {
15     ll n, m = 0;
16     vector<vector<ll>> adj;
17     vector<pair<ll, ll>> E;
18     vector<ll> deg, vertices, edges;
19
20     undirected_euler_trail(ll n) : n(n), adj(n), deg(n) {}
21
22     void add_edge(ll u, ll v) {
23         adj[u].push_back(m);
24         if (u != v) adj[v].push_back(m);
25         E.push_back({u, v});
26         deg[u]++, deg[v]++, m++;
27     }
28
29     ll find_start() {
30         ll s = -1, codd = 0;
31         for (ll v = 0; v < n; v++) {
32             if (deg[v]&1) codd++, s = v;
33             else if (deg[v] > 0 && s < 0)
34                 s = v;
35         }
36         return (codd == 0 || codd == 2) ? s : -2;
37     }
38
39     bool solve() {
40         if (m == 0) { vertices = {0}; return true; }
41
42         ll s = find_start();
43         if (s == -2) return false;
44
45         vector<ll> ep(n);
46         edges.resize(m);
47         vector<bool> alive(m, true);
48         ll pp = 0, pq = m, v = s;
49
50         while (pp < pq) {
51             if (ep[v] >= adj[v].size()) {
52                 if (pp == 0)
53                     return false;
54                 edges[--pq] = edges[--pp];
55                 v ^= E[edges[pq]].first ^ E[edges[pq]].second;
56             } else {
57                 ll e = adj[v][ep[v]++];
58                 if (!alive[e])
59                     continue;
60                 alive[e] = false;
61                 edges[pp++] = e;
62                 v ^= E[e].first ^ E[e].second;
63             }
64         }
65         vertices = {s};
66         for (ll e : edges) {
67             s ^= E[e].first ^ E[e].second;
68             vertices.push_back(s);
69         }
70         return true;
71     }
72 };

```

## CaminoNoDirigidoEuler.cpp

## 3.5 Tarjan

```

1  /*
2  *Description:* Finds bridges, articulation points, and
3  biconnected components
4  in an undirected graph using Tarjan's algorithm.
5  *Complexity:* $O(V + E)$
6  *Status:* Tested on CSES, Codeforces, UVA
7  */

```

```

7  struct tarjan {
8      struct edge { int u, v, comp, bridge; };
9      int n, nbc, T;
10     vector<vector<int>> adj;
11     vector<edge> e;
12     vector<int> disc, low, art, st;
13     tarjan(int n) : n(n), nbc(0), T(0), adj(n), disc(n,
14         -1), low(n), art(n, 0) {}
15     int add_edge(int u, int v) {
16         int i = e.size();
17         adj[u].push_back(i);
18         adj[v].push_back(i);
19         e.push_back({u, v, -1, false});
20         return i;
21     }
22     void dfs(int u, int pe) {
23         low[u] = disc[u] = T++;
24         for (int i : adj[u]) {
25             if (i == pe) continue;
26             int v = e[i].u ^ e[i].v ^ u;
27             if (disc[v] < 0) {
28                 st.push_back(i);
29                 dfs(v, i);
30                 if (low[v] > disc[u])
31                     e[i].bridge = true;
32                 if (low[v] >= disc[u]) {
33                     art[u]++;
34                     while (st.back() != i) {
35                         e[st.back()].comp = nbc;
36                         st.pop_back();
37                     }
38                     e[st.back()].comp = nbc;
39                     st.pop_back();
40                     nbc++;
41                 }
42                 low[u] = min(low[u], low[v]);
43             } else if (disc[v] < disc[u]) {
44                 st.push_back(i);
45                 low[u] = min(low[u], disc[v]);
46             }
47         }
48     }
49     void solve() {
50         for (int i = 0; i < n; i++)
51             if (disc[i] < 0) dfs(i, -1), art[i]--;
52     }
53     bool is_bridge(int i) { return e[i].bridge; }
54     bool is_art(int v) { return art[v] > 0; }
55     int component(int i) { return e[i].comp; }
56 };

```

## tarjan.cpp

## 3.6 tarjan SCC

```

1  /*
2  *Description:* Finds SCC on a graph
3  *Complexity:* $O(V+E)$
4  *Status:* Tested on CSES
5  */
6  struct tarjan {
7      int n, timer;
8      vector<bool> vis, ist;
9      vector<int> tin, low, st, cmp;
10     vector<vector<int>> scc;
11     void dfs(int u, vector<vector<int>> & adj){
12         vis[u] = ist[u] = 1;
13         st.push_back(u); tin[u] = low[u] = timer++;
14         for (auto v : adj[u]){
15             if (!vis[v]){
16                 dfs(v, adj);
17                 low[u] = min(low[u], low[v]);
18             } else if (ist[v]){
19                 low[u] = min(low[u], tin[v]);
20             }
21         }
22         if (low[u] == tin[u]){
23             scc.emplace_back(); int cur;
24             do {
25                 cur = st.back(); st.pop_back();
26                 ist[cur] = 0; scc.back().push_back(cur);
27                 cmp[cur] = (int)scc.size() - 1;
28             } while (cur != u);
29         }
30     }
31     tarjan(vector<vector<int>> & adj){
32         n = adj.size(); tin.resize(n); timer = 0; cmp.resize(n);
33         low.resize(n); vis.resize(n, 0); ist.resize(n, 0);
34         for (int i = 0; i < n; i++) if (!vis[i]) dfs(i, adj);
35     }
36 };

```

TarjanSCC.cpp

## 4 Estructuras de Datos

### 4.1 Segment Tree – $O(\log n)$

```

1  /*
2  *Description:* Range queries, build  $O(n)$ , query and
3  *              update  $O(\log n)$ , positions  $[0, n - 1]$ 
4  *Status:* Highly tested
5  */
6  template<class T, T m_(T, T)> struct segment_tree{
7      int n; vector<T> ST;
8      segment_tree(){}
9      segment_tree(vector<T> &a){
10         n = a.size(); ST.resize(n << 1);
11         for (int i=n;i<(n<<1);i++)ST[i]=a[i-n];
12         for (int i=n-1;i>0;i--)ST[i]=m_(ST[i<<1],ST[i<<1|1]);
13     }
14     void update(int pos, T val){ // replace with val
15         ST[pos += n] = val;
16         for (pos >>= 1; pos > 0; pos >>= 1)
17             ST[pos] = m_(ST[pos<<1], ST[pos<<1|1]);
18     }
19     T query(int l, int r){ // [l, r]
20         T ansL, ansR; bool hasL = 0, hasR = 0;
21         for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
22             if (l & 1)
23                 ansL=(hasL?m_(ansL,ST[l++]):ST[l++]),hasL=1;
24             if (r & 1)
25                 ansR=(hasR?m_(ST[--r],ansR):ST[--r]),hasR=1;
26         }
27         if (!hasL) return ansR; if (!hasR) return ansL;
28         return m_(ansL, ansR);
29     };
30 };
```

segtree.cpp

### 4.2 Sparse Table – $O(n \log n)$ build, $O(1)$ query

```

1
2  int min_op(int a, int b) {
3      return min(a, b);
4  }
5
6  template <typename T, T (*m_)(T, T)>
7  struct sparse_table {
8      int n = 0;
9      vector<vector<T>> table;
10
11      sparse_table() {}
12
13      sparse_table(const vector<T>& arr) {
14         n = arr.size();
15         if (n == 0) return;
16
17         int k = log2_floor(n) + 1;
18         table.assign(n, vector<T>(k));
19
20         for (int i = 0; i < n; i++)
21             table[i][0] = arr[i];
22
23         for (int j = 1; j < k; j++) {
24             for (int i = 0; i + (1LL << j) <= n; i++) {
25                 table[i][j] = m_(
26                     table[i][j - 1],
27                     table[i + (1LL << (j - 1))][j - 1]
28                 );
29             }
30         }
31     }
32
33     T query(int l, int r) {
34         assert(0 <= l && l <= r && r < n);
35
36         int k = log2_floor(r - l + 1);
37
38         return m_(
39             table[l][k],
40             table[r - (1LL << k) + 1][k]
41         );
42     }
43
44     int log2_floor(int x) {
```

```

45         return x ? 63 - __builtin_clzll(x) : -1;
46     }
47 };
48
```

sparse-table.cpp

### 4.3 Segment Tree Lazy – $O(\log n)$

```

1  /*
2  *Description:* Seg tree + range updates,
3  *              build  $O(n)$ , query/update  $O(\log n)$ 
4  *Status:* Highly tested
5  */
6  using T1 = long long;
7  using T2 = long long;
8
9  T1 merge_op(T1 a, T1 b){ return a + b; }
10
11 void pushUpd_op(T2& parent, T2& child,
12 int l, int r, int cl, int cr){
13     child += parent;
14 }
15
16 void applyUpd_op(T2& lazy, T1& val,
17 int l, int r){
18     val += (r - l + 1) * lazy;
19 }
20
21 template<
22 class T1, class T2,
23 T1 (*merge)(T1, T1),
24 void (*pushUpd)(T2&,T2&,int,int,int,int),
25 void (*applyUpd)(T2&, T1&, int, int)
26 >
27 struct segment_tree_lazy{
28     vector<T1> ST; vector<T2> lazy;
29     vector<bool> upd; int n;
30     void build(int i, int l, int r,
31 vector<T1>&values){
32         if(l == r){ ST[i] = values[l]; return; }
33         build(i<<1, l, (l+r)>>1, values);
34         build(i<<1|1, (l+r)/2+1, r, values);
35         ST[i] = merge(ST[i<<1], ST[i<<1|1]);
36     }
37     segment_tree_lazy() {}
38     segment_tree_lazy(vector<T1>&values){
39         n = values.size(); ST.resize(n<<2|3);
40         lazy.resize(n<<2|3);
41         upd.resize(n<<2|3, false);
42         build(1, 0, n-1, values);
43     }
44     void push(int i, int l, int r){
45         if(upd[i]){
46             applyUpd(lazy[i], ST[i], l, r);
47             if(l != r){
48                 pushUpd(lazy[i], lazy[i<<1],
49 l, r, l, (l+r)/2);
50                 pushUpd(lazy[i], lazy[i<<1|1],
51 l, r, (l+r)/2+1, r);
52                 upd[i<<1] = 1;
53                 upd[i<<1|1] = 1;
54             }
55             upd[i] = false;
56             lazy[i] = T2();
57         }
58     }
59     void update(int i, int l, int r,
60 int a, int b, T2 &u){
61         if(l >= a and r <= b){
62             pushUpd(u, lazy[i], a, b, l, r);
63             upd[i] = true;
64         }
65         push(i, l, r);
66         if(l > b or r < a) return;
67         if(l >= a and r <= b) return;
68         update(i<<1, l, (l+r)>>1, a, b, u);
69         update(i<<1|1, (l+r)/2+1, r, a, b, u);
70         ST[i] = merge(ST[i<<1], ST[i<<1|1]);
71     }
72     void update(int a, int b, T2 u){
73         if(a > b){
74             update(0, b, u);
75             update(a, n-1, u);
76             return;
77         }
78         update(1, 0, n-1, a, b, u);
79     }
80     T1 query(int i, int l, int r,
81 int a, int b){
82         push(i, l, r);
83         if(a <= l and r <= b) return ST[i];
84         int mid = (l+r) >> 1;
```

```

85     if(mid < a)
86         return query(i<<1|1, mid+1, r, a, b);
87     if(mid >= b)
88         return query(i<<1, l, mid, a, b);
89     return merge(
90         query(i<<1, l, mid, a, b),
91         query(i<<1|1, mid+1, r, a, b));
92 }
93 T1 query(int a, int b){
94     if(a > b)
95         return merge(query(a,n-1), query(0,b));
96     return query(1, 0, n-1, a, b);
97 }
98 };
99
100 int main(){
101     int n; cin >> n;
102     vector<long long> a(n);
103     for(int i = 0; i < n; i++) cin >> a[i];
104     segment_tree_lazy<long long, long long,
105     merge_op, pushUpd_op, applyUpd_op> st(a);
106     int q; cin >> q;
107     while(q--){
108         int type; cin >> type;
109         if(type == 1){
110             int l, r; long long val;
111             cin >> l >> r >> val;
112             st.update(l, r, val);
113         }else{
114             int l, r; cin >> l >> r;
115             cout << st.query(l, r) << '\n';
116         }
117     }
118 }
119

```

segment-tree-lazy.cpp

#### 4.4 Fenwick Tree – $O(\log n)$

```

1 struct fenwick_tree {
2     vector<ll> bit; int n;
3     fenwick_tree(int n): n(n) { bit.assign(n, 0); }
4     fenwick_tree(vector<ll> &a): fenwick_tree(a.size()) {
5         for (size_t i = 0; i < a.size(); i++)
6             add(i, a[i]);
7     }
8     ll sum(int r) {
9         ll ret = 0;
10        for (; r >= 0; r = (r & (r + 1)) - 1)
11            ret += bit[r];
12        return ret;
13    }
14    ll sum(int l, int r) {
15        return sum(r) - sum(l - 1);
16    }
17    void add(int idx, ll delta) {
18        for (; idx < n; idx = idx | (idx + 1))
19            bit[idx] += delta;
20    }
21 };
22

```

fenwick\_tree.cpp

```

1 // Returns the 0-based index of the k-th remaining
2 // element.
3 int fenwick_tree::kth(int k) {
4     int i = -1;
5     int s = 1;
6     while ((s << 1) <= n) s <<= 1;
7     for (; s > 0; s >>= 1) {
8         int j = i + s;
9         if (j < n && bit[j] < k) {
10             i = j;
11             k -= bit[j];
12         }
13     }
14     return i + 1;
15 }

```

kth\_fenwick.cpp

#### 4.5 Fenwick Tree 2D – $O(\log n)$

```

1
2 struct fenwick_tree_2d {
3     int N, M;
4     vector<vector<ll>> BIT;

```

```

5     fenwick_tree_2d(int N, int M): N(N), M(M) {
6         BIT.assign(N + 1, vector<ll>(M + 1, 0));
7     }
8     void update(int x, int y, ll v) {
9         for (int i = x; i <= N; i += (i & -i))
10            for (int j = y; j <= M; j += (j & -j))
11                BIT[i][j] += v;
12    }
13    ll sum(int x, int y) {
14        ll s = 0;
15        for (int i = x; i > 0; i -= (i & -i))
16            for (int j = y; j > 0; j -= (j & -j))
17                s += BIT[i][j];
18        return s;
19    }
20    ll query(int x1, int y1, int x2, int y2) {
21        return sum(x2, y2) - sum(x2, y1 - 1) - sum(x1 - 1,
22            y2) + sum(x1 - 1, y1 - 1);
23    }
24 };
25

```

fenwick\_tree\_2d.cpp

#### 4.6 Line container – $O(\log n)$

```

1 const ll INF = 1e15;
2 struct Line {
3     mutable ll a, b, c;
4     bool operator<(Line r) const {
5         return a < r.a;
6     }
7     bool operator<(ll x) const {
8         return c < x;
9     }
10 };
11 // dynamically insert 'a*x + b' lines and query for
12 // maximum
13 struct LineContainer: multiset< Line, less<>> {
14     ll div(ll a, ll b) {
15         return a / b - ((a ^ b) < 0 && a % b);
16     }
17     bool isect(iterator x, iterator y) {
18         if (y == end()) return x -> c = INF, 0;
19         if (x -> a == y -> a) x -> c = x -> b > y -> b ? INF
20             : -INF;
21         else x -> c = div(y -> b - x -> b, x -> a - y -> a);
22         return x -> c >= y -> c;
23     }
24     //a*x+b
25     void add(ll a, ll b) {
26         // a *= -1, b *= -1 // for min
27         auto z = insert({
28             a,
29             b,
30             0
31         }); y = z++, x = y;
32         while (isect(y, z)) z = erase(z);
33         if (x != begin() && isect(--x, y)) isect(x, y =
34             erase(y));
35         while ((y = x) != begin() && (--x) -> c >= y -> c)
36             isect(x, erase(y));
37     }
38     ll query(ll x) {
39         if (empty()) return -INF; // INF for min
40         auto l = * lower_bound(x);
41         return l.a * x + l.b; // for min
42     }
43 };
44
45 void solve()
46 {
47     ll n; cin >> n;
48     LineContainer lc;
49     for (ll k = 1; k <= n; k++) {
50         ll bi; cin >> bi;
51         lc.add(2*k, -(k*k + bi));
52     }
53     ll q; cin >> q;
54     for (ll k = 0; k < q; k++) {
55         ll t; cin >> t;
56         if (t == 1) {
57             ll x; cin >> x;
58             cout << -lc.query(x) + x*x << endl;
59         } else {
60             ll a, b; cin >> a >> b;
61             lc.add(2*a, -(a*a + b));
62         }

```

```

63 }
64 }
65

```

line\_container.cpp

```

11 }
12 return ans;
13 }
14

```

BinPow.cpp

## 4.7 Sqrt decomp — $O\sqrt{n}$

```

1 template <class T, T merge(T,T)>
2 struct sqrt_decomp {
3     int n,sq;
4     vector<T> st,bl;
5     sqrt_decomp(vector<T> & a){
6         n = a.size(); st.resize(n);
7         sq = sqrtl(n); bl.resize((n+sq-1)/sq);
8         for (int i = 0; i < n; i++){
9             st[i] = a[i];
10            bl[i/sq] = ((i%sq == 0)?st[i]:merge(bl[i/sq],st[i]
11            ));
12        }
13    }
14    void update(int pos, T val) {
15        st[pos] = val;
16        int block = pos/sq, lb = block*sq, rb = min(block*sq
17        +sq-1,n-1);
18        T vl;
19        for (int i = lb; i <= rb; i++)
20            vl = ((i == lb)?st[i]:merge(vl,st[i]));
21        bl[block] = vl;
22    }
23    T query(int l, int r){
24        T res; bool hasR = 0;
25        for (int i = l/sq; i <= r/sq; i++){
26            int lb = i*sq, rb = i*sq+sq-1;
27            if (l <= lb && rb <= r){
28                res = (hasR?merge(res,bl[i]):bl[i]), hasR = 1;
29                continue;
30            }
31            for (int j = max(l,lb); j <= min(rb,r); j++)
32                res = (hasR?merge(res,st[j]):st[j]), hasR = 1;
33        }
34        return res;
35    }
36 };
37

```

sqrt\_decomp.cpp

## 5 Matemáticas

### 5.1 Template

```

1 #include<bits/stdc++.h>
2
3 using namespace std;
4
5 #define ll long long
6 #define ld long double
7 #define cd complex<long double>
8 #define ull unsigned long long
9 #define for_(i, n) for(ll i = 0; i < n; ++i)
10 #define INF LLONG_MAX
11 #define NINF LLONG_MIN
12 #define i128 __int128
13 #define u128 __uint128_t
14 const double PI =
15     3.141592653589793238462643383279502884197169399375105;
16
17 const double EPS = 1e-18;
18
19 const ll mod = 1e9 + 7; //switch per case
20

```

template.cpp

### 5.2 BinPow

```

1 /*
2 Tested : Tested
3 */
4
5 ll binpow(ll a, ll b, ll m){
6     ll ans = 1;
7     while(b > 0){
8         if(b&1) ans = (ans*a)%m;
9         a=(a*a)%m;
10        b>>=1;
11    }
12 }
13

```

### 5.3 CatalanNumbers

```

1 //n-catalan number = (2n)!/((n+1)!n!)
2 ll getNCatalanNumber(ll n){
3     double ans = 1;
4     for(ll k = 2; k <= n; k++) ans *= (n+k)/k;
5     return (ll)(ans + 0.001);
6 }
7 //TESTED
8 vector<ll> getCatalanNumbers(ll n){
9     vector<ll> C(n+1, 0);
10    C[0] = C[1] = 1;
11    C[2] = 2;
12    for(ll i = 3; i <= n; ++i){
13        for(ll j = 0; j < i; ++j) C[i] += C[j]*C[i-j-1];
14    }
15    return C;
16 }
17

```

CatalanNumbers.cpp

### 5.4 ConvexHull

```

1
2 //TESTED
3 //Monotonic Change
4 struct point{
5     double x, y;
6 };
7 ll orientation(const point &a, const point &b,const
8     point &c){
9     double sign = a.x*(b.y - c.y) - b.x*(a.y - c.y) + c
10    .x*(a.y -b.y);
11    if(sign > 0) return 1;
12    else if( sign < 0) return -1;
13    else return 0;
14 }
15 bool cw(const point &a, const point &b,const point &c,
16     const bool &include_colider){
17     ll o = orientation(a, b, c);
18     return (o > 0 || include_colider && 0==0);
19 }
20 bool ccw(const point &a, const point &b,const point &c,
21     const bool &include_colider){
22     ll o = orientation(a, b, c);
23     return (o < 0 || include_colider && o==0);
24 }
25 void convex_hull(vector<point> &a, bool include_colider)
26 {
27     if(a.size()==1) return;
28     sort(a.begin(), a.end(), [](const point &p1, const
29     point &p2){return (p1.x==p2.x ? p1.y < p2.y : p1.x <
30     p2.x);});
31     point A = a[0], B = a[a.size()-1];
32     vector<point> upper, lower;
33     upper.push_back(A);
34     lower.push_back(A);
35     for(ll i = 1; i < a.size(); ++i){
36         if(i == a.size()-1 || cw(A, a[i], B,
37         include_colider)){
38             while(upper.size() >= 2 && !(cw(upper[upper.
39             size()-2], upper[upper.size()-1], a[i],
40             include_colider))) upper.pop_back();
41             upper.push_back(a[i]);
42         }
43         if(i == a.size()-1 || ccw(A, a[i], B,
44         include_colider)){
45             while(lower.size() >= 2 && !(ccw(lower[lower
46             .size()-2], lower[lower.size()-1], a[i],
47             include_colider))) lower.pop_back();
48             lower.push_back(a[i]);
49         }
50     }
51     if(include_colider && upper.size() == a.size()){
52         reverse(a.begin(), a.end());
53         return;
54     }
55     a.clear();
56     for(ll i = 0; i < upper.size(); ++i) a.push_back(
57     upper[i]);
58     for(ll i = lower.size()-2; i >= 1; --i) a.push_back(
59     lower[i]);
60 }
61

```

```
47 }
48
```

convexhull.cpp

## 5.5 Chinese Remainder Theorem

```
1 //TESTED (PD: for 2x2 system but should work for any nxn
  )
2 #include "gcdExtended.cpp"
3
4 ll mod_inv(ll a, ll m){
5     ll x, y;
6     ll g = gcdExtended(a, m, x, y);
7     x = (x%m + m)%m;
8     return x;
9 }
10
11 struct Congruence {
12     ll b, n;
13 };
14
15 ll chinese_remainder_theorem(vector<Congruence> const&
  congruences) {
16     ll N = 1;
17     for (auto const& congruence : congruences) N *=
  congruence.n;
18
19     ll x = 0;
20     for (auto const& congruence : congruences) {
21         ll b_i = congruence.b;
22         ll N_i = N / congruence.n;
23         ll invN_i = mod_inv(N_i, congruence.n);
24         i128 aux = (b_i*N_i)%N;
25         aux = (aux*invN_i)%N;
26         x = (ll) (x + aux)%N;
27     }
28     return x;
29 }
30
```

crt.cpp

## 5.6 Discrete Log

```
1
2 //Tested
3
4 ll discrete_log(ll a, ll b, ll m) {
5     a %= m, b %= m;
6     if (a == 0) return b == 0 ? 1 : -1;
7     ll k = 1, add = 0, g;
8     while ((g = gcd(a, m)) > 1) {
9         if (b == k)
10             return add;
11         if (b % g)
12             return -1;
13         b /= g, m /= g, ++add;
14         k = (k * 111 * a / g) % m;
15     }
16
17     ll n = sqrt(m) + 1;
18     ll an = 1;
19     for_(i, n) an = (an * 111 * a) % m;
20
21     unordered_map<ll, ll> f2inv;
22     for (ll q = 0, f2 = b; q <= n; ++q) {
23         f2inv[f2] = q;
24         f2 = (f2 * 111 * a) % m;
25     }
26
27     for (ll p = 1, f1 = k; p <= n; ++p) {
28         f1 = (f1 * 111 * an) % m;
29         if (f2inv.count(f1)){
30             ll x = n * p - f2inv[f1] + add;
31             return x;
32         }
33     }
34     return -1;
35 }
36
```

discretelog.cpp

## 5.7 Discrete Root

```
1
2 //TESTED
3
4 #include "gcd.cpp"
```

```
5 #include "bin_pow.cpp"
6 #include "tot_euler.cpp"
7 #include "primitive_root.cpp"
8
9 vector<ll> DiscreteRoot(ll n, ll k, ll a){
10     if(a==0) return {0};
11     ll g = generator_(n);
12
13     ll sq = (ll)sqrt(n) + 1;
14     vector<pair<ll, ll>> dec(sq);
15     for(ll i = 1; i <= sq; ++i)
16         dec[i-1] = {binpow(g, i * sq * k % (n - 1), n),
  i};
17     sort(dec.begin(), dec.end());
18     ll h0 = -1;
19     for_(i, sq) {
20         ll my = binpow(g, i * k % (n - 1), n) * a % n;
21         auto it = lower_bound(dec.begin(), dec.end(),
  make_pair(my, 0));
22         if (it != dec.end() && it->first == my) {
23             h0 = it->second * sq - i;
24             break;
25         }
26     }
27     if(h0 == -1) return {-1};
28
29     ll delta = (n-1)/gcd(k, n-1);
30     vector<ll> x;
31     for (ll h = h0 % delta; h < n-1; h += delta)
32         x.push_back(binpow(g, h, n));
33     sort(x.begin(), x.end());
34     return x;
35 }
36
```

DiscreteRoot.cpp

## 5.8 Erathostenes Sieve

```
1 //Tested
2
3 vector<bool> erathostenesSieve(ll n){
4     vector<bool> isprime(n+1, true);
5     isprime[0] = isprime[1] = false;
6     for(ll i = 2; i <= n; (i+=2 ? i++ : i+=2)){
7         if(isprime[i]){
8             for(ll j = i*i; j <= n; j+=i) isprime[j] = false;
9         }
10     }
11     return isprime;
12 }
13
```

erathostenes-sieve.cpp

## 5.9 Factorization using Pollard Rho

```
1
2 mt19937_64 rng(chrono::steady_clock::now().
  time_since_epoch().count());
3
4 //TESTED up to 1e18, run T(n) < 1e8
5
6 ll mult(ll a, ll b, ll m) {
7     return (__int128)a * b % m;
8 }
9
10 ll f(ll x, ll c, ll m) {
11     return (mult(x, x, m) + c) % m;
12 }
13
14 #include "gcd.cpp"
15
16 ll rho(ll n, ll x0=2, ll c=1) {
17     ll x = x0;
18     ll y = x0;
19     ll g = 1;
20     while (g == 1) {
21         x = f(x, c, n);
22         y = f(y, c, n);
23         y = f(y, c, n);
24         g = gcd(abs(x - y), n);
25     }
26     return g;
27 }
28
29 //algorithm for the factorization
30 ll brent(ll n, ll x0=2, ll c=1) {
31     ll x = x0;
32     ll g = 1;
33     ll xs, y;
```

```

34  ll m = 128;
35  ll l = 1;
36  ll iter = 0;
37  while (g == 1 && iter<1000) {
38      y = x;
39      ll q = 1;
40      for (ll i = 1; i < l; i++)
41          x = f(x, c, n);
42      ll k = 0;
43      while (k < l && g == 1) {
44          xs = x;
45          for (ll i = 0; i < m && i < l - k; i++) {
46              x = f(x, c, n);
47              q = mult(q, abs(y - x), n);
48          }
49          g = gcd(q, n);
50          k += m;
51      }
52      l *= 2;
53      iter++;
54  }
55  if (g == n) {
56      return n;
57  }
58  return g;
59 }
60 }
61
62 #include "miller_rabin.cpp"
63
64 //algorithm to prime factorization
65 void factorize(ll n, map<ll, ll> &factors){
66     if(n==1) return;
67     if(deterministicMillerRabin(n)){
68         factors[n]++;
69         return;
70     }
71     ll d1, d2, iter=0;
72     while(iter<1000){
73         uniform_int_distribution<ll> dist(1, n-1);
74         ll c = dist(rng);
75         ll x0 = dist(rng);
76         d1 = brent(n, x0, c), d2 = n/d1;
77         if(d1>1 && d1<n) break;
78         iter++;
79     }
80     if(d1==1 || d2==1){
81         factors[n]++;
82         return;
83     }
84     factorize(d1, factors);
85     factorize(d2, factors);
86 }
87

```

Factorization-pollard-rho.cpp

## 5.10 Fast Fourier Transform

```

1  //Tested
2
3  void fft_bottom_up(vector<cd> &c, bool inverse){
4      ll N = 1;
5      while(N < c.size()) N<=2;
6      c.resize(N);
7      if(N<= 1) return;
8      ll cnt = 0;
9      for(ll j = 1; j < N; ++j){
10         ll bit = N >> 1;
11         while(cnt & bit){
12             cnt ^= bit;
13             bit >>= 1;
14         }
15         cnt ^= bit;
16         if(cnt < j) swap(c[cnt], c[j]);
17     }
18
19     //n is the length of the current block of the tree,
20     //thanks to bit reversal is in order, n=2 -> depth= D
21     //n=4 -> depth=D-2, and so on...
22     for(ll n = 2; n <= N; n<=2){
23         double ang = 2*PI/n * (inverse ? -1 : 1);
24         cd wn(cos(ang), sin(ang));
25         //iterate each block(node from the current depth
26         for(ll currblock = 0; currblock < N; currblock+=
27             n){
28             cd w(1);
29             //iterate the node of the current block,
30             //only recover a half, because butterfly effect(como
31             //se diga la wea).
32             for(ll curr = 0; curr < n/2; ++curr){
33

```

```

28         cd u = c[currblock + curr], v = c[
29             currblock + curr + n/2]*w;
30         c[currblock + curr] = u + v;
31         c[currblock + curr + n/2] = u - v;
32         w*=wn;
33     }
34 }
35 if(inverse){
36     for(cd &x : c) x/=N;
37 }
38 }
39
40 vector<cd> conv(vector<cd> &c1, vector<cd> &c2){
41     vector<cd> ans; ll N=1;
42     while(N < (c1.size()+c2.size())) N<=2;
43     c1.resize(N); c2.resize(N);
44     fft_bottom_up(c1, false);
45     fft_bottom_up(c2, false);
46     ans.resize(N);
47     for_(i, N) ans[i] = c1[i]*c2[i];
48     fft_bottom_up(ans, true);
49     return ans;
50 }
51

```

fft.cpp

## 5.11 Gaussian Elimination

```

1  //Tested
2
3  bool gauss_jordan(vector<vector<ll>> A, vector<ll> &x) {
4      ll n = A.size(), m = A[0].size() - 1;
5
6      vector<ll> where(m, -1);
7      for (ll col = 0, row = 0; col < m && row < n; ++col) {
8          ll pivot = row;
9          for (ll i = row; i < n; ++i){
10             if (abs(A[i][col]) > abs(A[pivot][col])) pivot = i
11             ;
12             if(!A[pivot][col]) continue;
13             for (ll i = col; i <= m; ++i) swap(A[pivot][i], A[
14                 row][i]);
15             where[col] = row;
16
17             for(ll i = 0; i < n; ++i){
18                 ll invm = binpow(A[where[col]][col], MOD - 2, MOD);
19                 if(i != row) {
20                     ll c = (A[i][col] * invm) % MOD;
21                     for (ll j = col; j <= m; ++j) A[i][j] = ((A[i][j]
22                         - A[where[col]][j] * c) % MOD + MOD) % MOD;
23                 }
24             }
25             ++row;
26         }
27
28         x.assign(m, 0);
29         for (ll i = 0; i < m; ++i){
30             if (where[i] != -1){
31                 ll invm = binpow(A[where[i]][i], MOD - 2, MOD);
32                 x[i] = (A[where[i]][m] * invm) % MOD;
33             }
34         }
35         for(ll i=0; i<n; ++i){
36             ll sum = 0;
37             for(ll j=0; j<m; ++j) sum += (x[j] * A[i][j])%MOD;
38             if(abs(sum - A[i][m]) > 0) return false; //No
39             //solution
40         }
41     }
42     return true; //Valid solution, trivial or infinity
43 }
44

```

gaussian-elimination.cpp

## 5.12 GCD

```

1  //Tested meme
2  ll gcd(ll a, ll b){
3      if(b==0) return a;
4      return gcd(b, a%b);
5  }
6
7  ll gcd(ll a, ll b){
8      while(b) { ll r = a % b; a = b; b = r; }
9      return a;
10 }
11

```

gcd.cpp

## 5.13 GCD Extended

```

1 //Tested
2
3 ll gcdExtended(ll a, ll b, ll &x, ll &y){
4     if(b == 0){
5         x = 1;
6         y = 0;
7         return a;
8     }
9     ll x_, y_, gcd = gcdExtended(b, a%b, x_, y_);
10    x = y_;
11    y = x_ - y_*(a/b);
12    return gcd;
13 }
14

```

gcdExtended.cpp

## 5.14 Geometry Structures

```

1
2 struct p2d {
3     ll x, y;
4     p2d() {}
5     p2d(ll x, ll y): x(x), y(y) {}
6     p2d& operator+=(const p2d &p) {
7         x += p.x;
8         y += p.y;
9         return *this;
10    }
11    p2d& operator-=(const p2d &p) {
12        x -= p.x;
13        y -= p.y;
14        return *this;
15    }
16    p2d& operator+=(ll c) {
17        x += c;
18        y += c;
19        return *this;
20    }
21    p2d& operator-=(ll c) {
22        x -= c;
23        y -= c;
24        return *this;
25    }
26    p2d operator+(const p2d &p) const {
27        return p2d(*this) += p;
28    }
29    p2d operator-(const p2d &p) const {
30        return p2d(*this) -= p;
31    }
32    p2d operator*(ll c) const {
33        return p2d(*this) *= c;
34    }
35    p2d operator/(ll c) const {
36        return p2d(*this) /= c;
37    }
38    ll dot(p2d a, p2d b){
39        return a.x*b.x + a.y*b.y;
40    }
41    double norm2(p2d a) {
42        return sqrt(dot(a, a));
43    }
44    double norm2sq(p2d a) {
45        return dot(a, a);
46    }
47    p2d proj(p2d a, p2d b) {
48        return b*(dot(a, b) / norm2sq(b));
49    }
50    double angle(p2d a, p2d b) {
51        return acos(dot(a, b) / norm2(a) / norm2(b));
52    }
53    static ll cross(p2d a, p2d b) {
54        return a.x * b.y - a.y * b.x;
55    }
56    p2d intersect_lines(p2d a1, p2d d1, p2d a2, p2d d2){
57        ll crs = cross(d1, d2);
58        if(crs==0) return p2d(INF, INF);
59        return a1 + d1*(cross(a2 - a1, d2) / crs);
60    }
61 };
62
63 struct p3d {
64     ll x, y, z;
65     p3d() {}
66     p3d(ll x, ll y, ll z): x(x), y(y), z(z) {}
67     p3d& operator+=(const p3d &p) {
68         x += p.x;
69         y += p.y;
70         z += p.z;
71         return *this;
72    }
73

```

```

73    p3d& operator-=(const p3d &p) {
74        x -= p.x;
75        y -= p.y;
76        z -= p.z;
77        return *this;
78    }
79    p3d& operator+=(ll c) {
80        x += c;
81        y += c;
82        z += c;
83        return *this;
84    }
85    p3d& operator/(ll c) {
86        x /= c;
87        y /= c;
88        z /= c;
89        return *this;
90    }
91    p3d operator+(const p3d &p) const {
92        return p3d(*this) += p;
93    }
94    p3d operator-(const p3d &p) const {
95        return p3d(*this) -= p;
96    }
97    p3d operator*(ll c) const {
98        return p3d(*this) *= c;
99    }
100    p3d operator/(ll c) const {
101        return p3d(*this) /= c;
102    }
103    ll dot(p3d a, p3d b){
104        return a.x*b.x + a.y*b.y + a.z*b.z;
105    }
106    double norm2(p3d a) {
107        return sqrt(dot(a, a));
108    }
109    double norm2sq(p3d a) {
110        return dot(a, a);
111    }
112    p3d proj(p3d a, p3d b) {
113        return b*(dot(a, b) / norm2sq(b));
114    }
115    double angle(p3d a, p3d b) {
116        return acos(dot(a, b) / norm2(a) / norm2(b));
117    }
118    p3d cross(p3d a, p3d b) {
119        return p3d(a.y * b.z - a.z * b.y,
120                a.z * b.x - a.x * b.z,
121                a.x * b.y - a.y * b.x);
122    }
123    ll triple(p3d a, p3d b, p3d c) {
124        return dot(a, cross(b, c));
125    }
126
127    p3d intersect_planes(p3d a1, p3d n1, p3d a2, p3d n2,
128        p3d a3, p3d n3){
129        p3d x(n1.x, n2.x, n3.x);
130        p3d y(n1.y, n2.y, n3.y);
131        p3d z(n1.z, n2.z, n3.z);
132        p3d d(dot(a1, n1), dot(a2, n2), dot(a3, n3));
133        return p3d(triple(d, y, z), triple(x, d, z),
134            triple(x, y, d)) / triple(n1, n2, n3);
135    }
136 };
137 //TESTED
138 //orientation of triangle
139 ll sign(const p2d& a, const p2d& b, const p2d& c){
140     ll val = p2d::cross(b - a, c - a);
141     if (val > 0) return 1;
142     if (val < 0) return -1;
143     return 0;
144 }
145 //TESTED
146 bool inter1(ll a, ll b, ll c, ll d) {
147     if (a > b) swap(a, b);
148     if (c > d) swap(c, d);
149     return max(a, c) <= min(b, d);
150 }
151 //TESTED
152 bool intersect_segments(const p2d& a, const p2d& b,
153     const p2d& c, const p2d& d) {
154     if (sign(a, b, c) == 0 && sign(a, b, d) == 0) return
155         inter1(a.x, b.x, c.x, d.x) && inter1(a.y, b.y, c.y,
156             d.y);
157     ll ori1 = sign(a, b, c);
158     ll ori2 = sign(a, b, d);
159     ll ori3 = sign(c, d, a);
160     ll ori4 = sign(c, d, b);
161     return (ori1 != ori2) && (ori3 != ori4);
162 }
163

```

GeometryStructures.cpp

## 5.15 Inclusion Exclusion

```

1
2 //Example cardinality of all subsets of size 1
3 ll cardinalitySubset(ll subset){
4     return 1;
5 }
6
7 //Template
8 ll InclusionExclusion(){
9     //sets
10    ll n = 10; //example
11    ll subsets = (1<n); //total subsets
12    ll ans = 0;
13    //travel on all the subsets, with a particular
    subset
14    for(ll subset = 0; subset < subsets; subset++){
15        //if is odd the subset we sum, otherwise
        subtract, recall, Union of sets = sum sets -
        intersect sets pairs + intersect group of three -
        ...
16        ans += (__builtin_popcount(subset)&1 ?
        cardinalitySubset(subset) : -cardinalitySubset(
        subset));
17    }
18    return ans;
19 }
20

```

InclusionExclusion.cpp

## 5.16 Miller Rabin

```

1
2 ll binpow(ll a, ll b, ll mod){
3     ll ans = 1;
4     while(b>0){
5         if(b&1) ans = (u128)(ans)*a % mod;
6         a=(u128)a*a % mod;
7         b>>=1;
8     }
9     return ans;
10 }
11
12 bool checkComposite(ll p, ll a, ll s, ll q){
13     ll x = binpow(a, q, p);
14     if(x == 1 || x == p-1) return false;
15     for(ll i = 1; i < s; ++i){
16         x = (u128)x*x % p;
17         if(x == p-1) return false;
18     }
19     return true;
20 }
21
22 bool millerRabin(ll p, ll maxIter=5){
23     if(p<4) return p==2 || p==3;
24     ll s=0, q = p-1;
25     while(!(q&1)){
26         s++;
27         q>>=1;
28     }
29     for(ll i = 0; i < maxIter; ++i){
30         ll a = 2 + rand() % (p-3);
31         if(checkComposite(p, a, s, q)) return false;
32     }
33     return true;
34 }
35
36 bool deterministicMillerRabin(ll p){
37     if(p<4) return p==2 || p==3;
38     ll s=0, q = p-1;
39     while(!(q&1)){
40         s++;
41         q>>=1;
42     }
43     for(ll a : {2,3,5,7,11,13,17,19,23,29,31,37}){
44         if(p==a) return true;
45         if(checkComposite(p, a, s, q)) return false;
46     }
47     return true;
48 }
49

```

miller-rabin.cpp

## 5.17 Mobius

```

1 //Tested
2
3 vector<ll> mobius(ll n){
4     vector<ll> mob(n+1);
5

```

```

5     mob[1] = 1;
6     vector<bool> isprime(n+1, true);
7     isprime[0]=isprime[1] = false;
8     for(ll i = 2; i < n+1; i++){
9         if(isprime[i]){
10            for(ll j = i; j < n+1; j+=i){
11                mob[j] += -1;
12                isprime[j] = false;
13            }
14            for(ll j = i*i; j < n+1; j += i*i) mob[j] =
15                0;
16        }
17    }
18    return mob;
19 }

```

mobius.cpp

## 5.18 Number Theoretic Transform

```

1
2 //TESTED
3
4 const ll mod = 998244353, generator = 3, rootn = 3; //
    for N max = 2^23
5 /*more modulus primes of the form p = c*2^m + 1 (
    generators calculated with algorithm in math section
    )
6 const ll mod = 7340033, generator = 3, rootn = 3; // for
    N max = 2^20
7 const ll mod = 167772161, generator = 3, rootn = 3; //
    for N max = 2^25
8 const ll mod = 469762049, generator = 3, rootn = 3; //
    for N max = 2^26
9 */
10
11 ll binpow(ll a, ll e){
12     ll ans = 1;
13     while(e > 0){
14         if(e&1) ans = (ans * a)%mod;
15         a = (a*a)%mod;
16         e>>=1;
17     }
18     return ans;
19 }
20
21 ll inv(ll a){return binpow(a, mod-2);}
22
23
24 void ntt(vector<ll> &c, bool invert){
25     ll N = 1;
26     while(N < c.size())N<=1;
27     c.resize(N);
28     ll cnt = 0;
29     for(ll j = 1; j < N; ++j){
30         ll bit = N>>1;
31         while(bit&cnt){
32             cnt^=bit;
33             bit>>=1;
34         }
35         cnt^=bit;
36         if(cnt < j) swap(c[cnt], c[j]);
37     }
38
39     for(ll n = 2; n <= N; n<=1){
40         ll wn = binpow(generator, (mod-1)/n);
41         if(invert) wn = inv(wn);
42         for(ll currblock = 0; currblock < N; currblock+=
43             n){
44             ll w = 1;
45             for(ll i = 0; i < n/2; ++i){
46                 ll u = c[currblock + i]%mod, v = (w*c[
47                     currblock + i + n/2])%mod;
48                 c[currblock + i] = (u + v)%mod;
49                 c[currblock + i + n/2] = ((u - v)%mod +
50                     mod)%mod;
51                 w = (w*wn)%mod;
52             }
53         }
54     }
55     if(invert){
56         for(ll &x : c) x = (x*inv(N))%mod;
57     }
58 }
59
60 vector<ll> conv(vector<ll> c1, vector<ll> c2){
61     ll N=1;
62     while(N <= (c1.size() + c2.size())) N<=1;
63     c1.resize(N); c2.resize(N);
64     ntt(c1, false);
65     ntt(c2, false);
66     vector<ll> ans(N, 0);
67

```

```

64     for_(i, N) ans[i] = (c1[i] * c2[i])%mod;
65     ntt(ans, true);
66     return ans;
67 }
68

```

ntt.cpp

## 5.19 Primitive Root (Generator of a Prime Group)

```

1 //TESTED
2
3 #include "bin_pow.cpp"
4 #include "tot_euler.cpp"
5
6
7 ll generator(ll n) {
8     vector<int> facts;
9     ll phi_n = tot_euler(n)[n], m = phi_n;
10    for(ll i=2; i*i<=m; ++i){
11        if(m % i == 0) {
12            facts.push_back(i);
13            while (m % i == 0) m/= i;
14        }
15    }
16    if(m > 1) facts.push_back(m);
17    for (ll g=2; g<=n; ++g) {
18        bool ok = true;
19        for (size_t i=0; i<facts.size() && ok; ++i) ok
20            &= binpow(g, phi_n/facts[i], n) != 1;
21        if (ok) return g;
22    }
23    return -1;
24 }

```

primitive-root.cpp

## 5.20 Simpson Rule

```

1 //TESTED
2
3 ld f(ld x) {return x;}
4
5 ld simpson_integration(ld a, ld b, ld d){
6     const ll N = 10000; // number of steps (already
7         multiplied by 2)
8     ld h = (b - a) / N;
9     ld s = f(a) + f(b); // a = x_0 and b = x_2n
10    for(ll i=1; i < N; i++) { // Refer to final Simpson
11        //s formula
12        double xi = a + h * i;
13        s += f(xi) * ((i & 1) ? 4 : 2);
14    }
15    s *= h / 3.0;
16    //1e6 is the decimal precision, if you want 7, the 1
17    e7 and so on
18    s*=1e6;
19    s=llround(s);
20    s/=1e6;
21    // to print use cout << fixed << setprecision(6) <<
22    s;
23    return s;
24 }

```

SimpsonRule.cpp

## 5.21 Tonelli Shanks

```

1 // TESTED
2
3 #include "bin_pow.cpp"
4
5 ll TonelliShanks(ll n, ll p) {
6     n %= p; if (n < 0) n += p;
7     if (n == 0) return 0;
8     if (binpow(n, (p-1)/2, p) != 1) return -1; // solution
9         does not exist
10    if (p % 4 == 3) return binpow(n, (p + 1) / 4, p); //
11        trivial solution, only one factor 2
12    ll c = p - 1, z = 2;
13    ll s = 0;
14    while(c % 2 == 0) ++s, c /= 2;
15    // find a non-square mod p
16    while(binpow(z, (p-1)/2, p) != p - 1) ++z;
17    ll q = binpow(z, c, p); // generator subgroup, |<q>|
18    = 2^s

```

```

16     ll a = binpow(n, (c+1)/2, p); // candidate root
17     ll t = binpow(n, c, p); // error from equation a^2 =
18         n*t, if t=1, a is the root
19     ll m = s;
20     while(1){
21         ll i = 0, temp = t;
22         while(i < m && temp != 1){
23             temp = (temp*temp)%p;
24             i++;
25         }
26         if(i==0) return a;
27         ll exponent = 1ll << (m-i-1);
28         ll b = binpow(q, exponent, p);
29         a = (a*b)%p;
30         t = (((t*b)%p)*b)%p;
31         q = (b*b)%p;
32         m=i;
33     }
34 }

```

TonelliShanks.cpp

## 5.22 Euler Totiente

```

1 //Tested
2
3 vector<ll> tot_euler(ll n){
4     vector<ll> tot(n+1, 0);
5     for_(i, n+1) tot[i]=i;
6     for(ll i = 2; i <= n; ++i){
7         if(tot[i] == i){
8             for(ll j = i; j <= n; j+=i) tot[j] -= tot[j]
9                 /i;
10        }
11    }
12    return tot;
13 }

```

tot-euler.cpp

## 6 Strings

### 6.1 KMP – $O(n + m)$

```

1 /*
2 *Description:* Pattern matching in  $O(n+m)$ 
3 *Status:* Tested on CSES
4 */
5 template<class T> struct KMP {
6     T pattern; vector<int> lps;
7     KMP(T &pat): pattern(pat) {
8         lps.resize(pat.size(), 0);
9         int len = 0, i = 1;
10        while (i < pattern.size()) {
11            if (pattern[i] == pattern[len])
12                lps[i++] = ++len;
13            else {
14                if (len != 0) len = lps[len-1];
15                else lps[i++] = 0;
16            }
17        }
18    }
19    vector<int> search(T &text) {
20        vector<int> matches;
21        int i = 0, j = 0;
22        while (i < text.size()) {
23            if (pattern[j] == text[i]) {
24                i++, j++;
25                if (j == pattern.size()) {
26                    matches.push_back(i - j);
27                    j = lps[j - 1];
28                }
29            } else {
30                if (j != 0) j = lps[j-1];
31                else i++;
32            }
33        }
34        return matches;
35    }
36 };
37
38 // Prefix function (simple version)
39 template<class T>
40 vector<int> prefix(T &S) {
41     vector<int> P(S.size());
42     P[0] = 0;
43     for(int i = 1; i < S.size(); ++i) {
44         P[i] = P[i - 1];

```

```

45     while(P[i] > 0 && S[P[i]] != S[i])
46         P[i] = P[P[i] - 1];
47     if(S[P[i]] == S[i])
48         ++P[i];
49 }
50 return P;
51 }
52
53 int main(){
54     string text, pattern;
55     cin >> text >> pattern;
56     KMP<string> kmp(pattern);
57     vector<int> matches = kmp.search(text);
58     cout << matches.size() << '\n';
59     for(int x : matches) cout << x << " ";
60     cout << '\n';
61 }
62

```

KMP.cpp

## 6.2 Z-Function – $O(n + m)$

```

1  /*
2  *Description:* Z-algorithm, all occurrences
3  in  $O(n+m)$ 
4  *Status:* Tested
5  */
6  struct Z {
7      int n, m;
8      vector<int> z;
9      Z(string s) {
10         n = s.size();
11         z.assign(n, 0);
12         int l = 0, r = 0;
13         for (int i = 1; i < n; i++) {
14             if (i <= r)
15                 z[i] = min(r-i+1, z[i-l]);
16             while (i+z[i] < n
17                 && s[z[i]] == s[i+z[i]])
18                 ++z[i];
19             if (i+z[i]-1 > r)
20                 l = i, r = i+z[i]-1;
21         }
22     }
23     Z(string p, string t) {
24         string s = p + "#" + t;
25         n = p.size();
26         m = t.size();
27         z.assign(n+m+1, 0);
28         int l = 0, r = 0;
29         for (int i = 1; i < n+m+1; i++) {
30             if (i <= r)
31                 z[i] = min(r-i+1, z[i-l]);
32             while (i+z[i] < n+m+1
33                 && s[z[i]] == s[i+z[i]])
34                 ++z[i];
35             if (i+z[i]-1 > r)
36                 l = i, r = i+z[i]-1;
37         }
38     }
39     void p_in_t(vector<int>& ans) {
40         for (int i = n+1; i < n+m+1; i++)
41             if (z[i] == n)
42                 ans.push_back(i-n-1);
43     }
44 };
45
46 int main(){
47     string t, p;
48     cin >> t >> p;
49     Z z(p, t);
50     vector<int> ans;
51     z.p_in_t(ans);
52     cout << ans.size() << '\n';
53     for(int x : ans) cout << x << " ";
54     cout << '\n';
55 }
56

```

z-function.cpp

## 6.3 Trie – $O(|s|)$ por operación

```

1  /*
2  *Description:* Trie for prefix queries.
3   $O(|s|)$  per operation.
4  *Status:* Tested
5  */
6  struct trie {
7      vector<vector<ll>> tree;

```

```

8      vector<ll> counts;
9      trie() {
10         tree.push_back(vector<ll>(26, -1));
11         counts.push_back(0);
12     };
13     void insert(string &s) {
14         ll i = 0, u = 0;
15         while(i < s.size()){
16             char c = s[i];
17             if (tree[u][c-'a'] == -1){
18                 ll pos = tree.size();
19                 tree.push_back(vector<ll>(26, -1));
20                 counts.push_back(0);
21                 tree[u][c-'a'] = pos;
22             }
23             i++;
24             u = tree[u][c-'a'];
25         }
26         counts[u]++;
27     }
28     ll search(string &s){
29         ll i = 0, u = 0;
30         ll count_ends = 0;
31         while(i < s.size()){
32             char c = s[i];
33             if (tree[u][c-'a'] != -1){
34                 u = tree[u][c-'a'];
35                 i++;
36             }else break;
37         }
38         if(i == s.size())
39             count_ends += counts[u];
40         return count_ends;
41     }
42 };
43
44 int main(){
45     int n; cin >> n;
46     trie t;
47     for(int i = 0; i < n; i++){
48         string s; cin >> s;
49         t.insert(s);
50     }
51     int q; cin >> q;
52     while(q--){
53         string s; cin >> s;
54         cout << t.search(s) << '\n';
55     }
56 }
57

```

trie.cpp

## 6.4 Rolling Hashing

```

1  template<class T>
2  struct rolling_hashing {
3      int base, mod;
4      vector<long long> p, H;
5      int n;
6      rolling_hashing(const T &s, int b, int m): base(b),
7          mod(m), n(s.size()) {
8          p.assign(n+1, 1);
9          H.assign(n+1, 0);
10         for (int i = 0; i < n; ++i) {
11             H[i+1] = (H[i] * base + s[i]) % mod;
12             p[i+1] = (p[i] * base) % mod;
13         }
14     }
15     int get(int l, int r) {
16         int res = (H[r+1] - H[l]*p[r-l+1]) % mod;
17         if (res < 0) res += mod;
18         return res;
19     };
20 };

```

rolling\_hashing.cpp

## 6.5 aho-corasick

```

1  /*
2  *Description:* Aho-Corasick algorithm for multiple
3  pattern matching in text
4  *Status:* Tested
5  */
6  struct AhoCorasick {
7      enum {
8          alpha = 26, first = 'a'
9      }; // change this!
10     struct Node {

```

```

10 // (nmatches is optional)
11 int back, next[alpha], start = -1, end = -1,
12   nmatches = 0;
13 Node(int v) {
14     memset(next, v, sizeof(next));
15 }
16 vector < Node > N;
17 vi backp;
18 void insert(string & s, int j) {
19     assert(!s.empty());
20     int n = 0;
21     for (char c: s) {
22         int & m = N[n].next[c - first];
23         if (m == -1) {
24             n = m = sz(N);
25             N.emplace_back(-1);
26         } else n = m;
27     }
28     if (N[n].end == -1) N[n].start = j;
29     backp.push_back(N[n].end);
30     N[n].end = j;
31     N[n].nmatches++;
32 }
33 AhoCorasick(vector < string > & pat): N(1, -1) {
34     rep(i, 0, sz(pat)) insert(pat[i], i);
35     N[0].back = sz(N);
36     N.emplace_back(0);
37
38     queue < int > q;
39     for (q.push(0); !q.empty(); q.pop()) {
40         int n = q.front(), prev = N[n].back;
41         rep(i, 0, alpha) {
42             int & ed = N[n].next[i], y = N[prev].next[i];
43             if (ed == -1) ed = y;
44             else {
45                 N[ed].back = y;
46                 (N[ed].end == -1 ? N[ed].end : backp[N[ed].
47 start]) = N[y].end;
48                 N[ed].nmatches += N[y].nmatches;
49                 q.push(ed);
50             }
51         }
52     }
53
54     vi find(string word) {
55         int n = 0;
56         vi res; // ll count = 0;
57         for (char c: word) {
58             n = N[n].next[c - first];
59             res.push_back(N[n].end);
60             // count += N[n].nmatches;
61         }
62         return res;
63     }
64
65     vector < vi > findAll(vector < string > & pat, string
66 word) {
67         vi r = find(word);
68         vector < vi > res(sz(word));
69         rep(i, 0, sz(word)) {
70             int ind = r[i];
71             while (ind != -1) {
72                 res[i - sz(pat[ind]) + 1].push_back(ind);
73                 ind = backp[ind];
74             }
75         }
76     }
77 };

```

aho-corasick.cpp

## 7 Programación Dinámica

### 7.1 Longest Increasing Subsequence – $O(n \log n)$

```

1 /*
2  *Description:* LIS in  $O(n \log n)$ ,
3  *allows sequence recovery
4  *Status:* Highly tested
5  */
6 template <class I>
7 vector<int> LIS(const vector<I> &S) {
8     if (S.empty()) return {};
9     vector<int> prev(S.size());
10    vector<pair<I, int>> res;
11    for (int i = 0; i < S.size(); i++) {
12        auto it = lower_bound(res.begin(),
13 res.end(), pair<I, int>{S[i], i});

```

```

14     if (it == res.end())
15         res.emplace_back(), it = res.end() - 1;
16     *it = {S[i], i};
17     prev[i] = (it == res.begin()
18 ? 0 : (it - 1) ->second);
19 }
20 int L = res.size(),
21 cur = res.back().second;
22 vector<int> ans(L);
23 while (L--)
24     ans[L] = cur, cur = prev[cur];
25 return ans;
26 }
27
28 int main(){
29     int n; cin >> n;
30     vector<int> a(n);
31     for(int i = 0; i < n; i++) cin >> a[i];
32     vector<int> lis_idx = LIS(a);
33     cout << lis_idx.size() << '\n';
34     for(int i = 0; i < (int)lis_idx.size(); i++)
35         cout << lis_idx[i] << " ";
36     cout << '\n';
37 }
38

```

LIS.cpp

### 7.2 LIS of length K– $O(nk \log n)$

```

1 //k = searched length, m = compressed size
2 vector<fenwick_tree> bit(k + 1, fenwick_tree(m));
3
4 ll answer = 0;
5 for (ll x : v) {
6     ll value = compress(x); //or already compressed
7     vector<ll> current(k+1);
8     current[1] = 1;
9     for (ll len = 2; len <= k; len++) {
10         current[len] = bit[len-1].sum(value-1);
11     }
12     for (ll len = 1; len <= k; len++) {
13         bit[len].add(value, current[len]);
14     }
15     answer += current[k];
16 }
17
18 cout<<answer<<endl;
19

```

LIS\_lenght\_k.cpp

### 7.3 LIS elements participate of Optimal solutions– $O(n \log n)$

```

1 /*
2  L[i]: longest strictly increasing subsequence ending
3  at i
4  R[i]: longest strictly increasing subsequence
5  starting at i
6  Theroem: best sol force through x = best before x+
7  best after x-overlap on x.
8  So if optimal=L[i]+R[i]-1(overlap) -> index i its
9  part of an optimal solution
10 layer: number of LIS-participating elements
11 */
12 fenwick_max<ll> bit(m + 1);
13 fenwick_max<ll> bit2(m + 1);
14
15 vector<ll> L(n), R(n);
16
17 for (ll i = 0; i < n; i++) {
18     L[i] = bit.query(v[i] - 1) + 1;
19     bit.update(v[i], L[i]);
20 }
21
22 for (ll i = n - 1; i >= 0; i--) {
23     R[i] = bit2.query(m - v[i]) + 1;
24     bit2.update(m - v[i] + 1, R[i]);
25 }
26
27 ll k = *max_element(L.begin(), L.end());
28
29 vector<ll> layer(k + 1, 0);
30
31 for (ll i = 0; i < n; i++) {
32     if (L[i] + R[i] - 1 == k) layer[L[i]]++;
33 }
34

```

LIS\_elements\_optimal.cpp

## 7.4 Knapsack 0/1 – $O(nW)$

```
1 void solve(){
2     ll n, w;
3     cin >> n >> w;
4     vector<pair<ll,ll>> v(n);
5     for(auto &a,b] : v) cin >> a >> b;
6     vector<ll> ans(w+2,0);
7     ll ans_ = 0;
8     for(auto [vi, wi] : v){
9         for(ll j = w; j >= wi; j--){
10             ans[j] = max(ans[j], ans[j-wi]+vi);
11             ans_ = max(ans_, ans[j]);
12         }
13     }
14     cout << ans_ << endl;
15 }
16
```

mochila.cpp

## 8 Misceláneos

### 8.1 Búsqueda Binaria – $O(\log n)$

```
1 int bs(vector<ll>& v, int b){
2     int l = 0, r = v.size();
3     while(l < r){
4         int mid = (l + r) / 2;
5         if(v[mid] >= b) r = mid;
6         else l = mid + 1;
7     }
8     return l;
9 }
10
```

binary search.cpp